

CMP5332

Object-Oriented Programming

Object-Oriented Modelling





Lecture Outlines

- What is a Class?
- What is a Method?
- Creating Software Models of Real World Entities
- Information Hiding (Encapsulation)
- Access Controls
- The use `Private` and `Final` Modifiers
- Examples of classes to implement based on a given specification:
 - User Account class
 - Bank Account class

What is a class?

A class can be thought of as **many things**:

- A grouping of related data.
- A grouping of related methods.
- A data structure.
- A data type.
- A “blueprint” for a kind of object.
- A way of hiding unnecessary details from other parts of a program, to simplify software development and maintenance.
- A model of some kind of entity.

```
class Person {  
    String name;  
    int age;  
  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    String getName() {  
        return name;  
    }  
  
    int getAge() {  
        return age;  
    }  
  
    void haveBirthday() {  
        age++;  
    }  
}
```

All of these are good ways of thinking about classes, but object-oriented programming is really about the last few — “**blueprints**”, **hiding unnecessary details**, and **modelling entities**.

What is a Method?

There are at least **two ways** to think about methods:

- A method is a **function** or **procedure** which belongs to an object, which does some computation with the object's fields.
- A method is a **way of interacting** with an object, asking it a question or telling it to do something.

```
class Person {  
    String name;  
    int age;  
  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    String getName() {  
        return name;  
    }  
  
    int getAge() {  
        return age;  
    }  
  
    void haveBirthday() {  
        age++;  
    }  
}
```

Both ideas are important, but the “**way of interacting**” idea is the object-oriented mode of thinking.

A method represents some **behaviour that the object has**, such as the way it should answer a question or fulfill an instruction.

An entity is a “**thing**”:

It could be something in the **real world** (like a person, a car, or a chess piece)

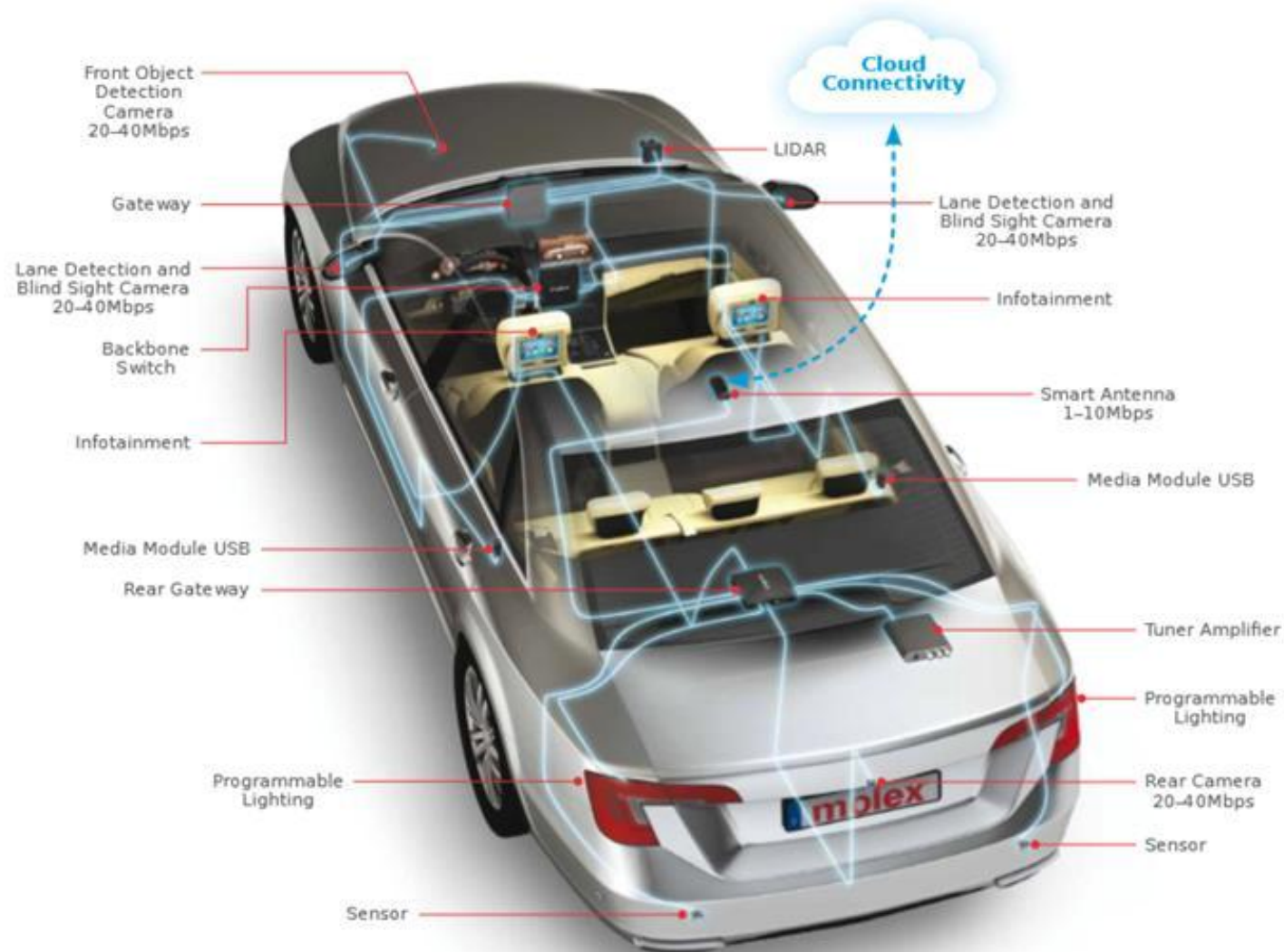
or

something **abstract** (like a rectangle, a vector, a bank account, user system login details).

In object-oriented programming, **a class is a model of an entity**.

A model is an **abstract (simplified)** representation of the entity, that *captures only the aspects which are relevant to the application* — for example, **car buyer** may need to know a car’s **make, model** and **mileage**, and **price** but not the engineering details.

Abstraction



A **car engineer** may need to know all these details

Encapsulation

For example, a capsule is a wrap on different medicines.

In OOP encapsulation is binding (or *wrapping*) code and data together into a single unit.



Access Control

Objects should be **encapsulated**, and only interacted with via their methods.

This means code in one class **should never directly access a field** belonging to an object of a different class. We can ask the compiler to **enforce this rule** using the **access modifiers controls**:

- ◆ **public**

Members declared as public are accessible from any other class.

- ◆ **private**

Members declared as private are only accessible from the class itself.

- ◆ **protected**

Only subclasses of the class and classes in the same package can create instances of it.

A variable or a method that is **declared without modifier** can be called by any other class **in the same package (public to the package)**.

Access Control

Access modifiers can apply to **classes**, **fields**, **methods** and **constructors**.

Whichever access modifier you use, you should **write it at the start** of the declaration.

Making classes, methods and constructors **public** will become more important later, when we start organising classes into different “**packages**” – in that case, when you try to access something from another package, you’ll get a compilation error if it’s not **public**.

For now, while all our classes are in the same package, it’s still a **good habit** to label things **public** if you do intend them to be used by other classes.

Modifier	Class	Package	Subclass	Global
Public	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	No
Default	Yes	Yes	No	No
Private	Yes	No	No	No

Access Modifier Controls

Consider the `Person` class example:

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    String getName() {
        return name;
    }

    int getAge() {
        return age;
    }

    void haveBirthday() {
        age++;
    }
}
```

By declaring the fields `name` and `age` as `private`, if we attempt to access them from outside the `Person` class then we now get a compilation error:

```
Person alice = new Person("Alice", 19);
```

Create a `PersonMain` class and see if you can access the attributes directly. Set the `name` and `age` attributes to `private`.

Access Controls

The private **access modifier** has **no effect** on what the `Person` class does — it is purely a constraint on what other classes can do with `Person` objects.

To **make explicit** that other classes are allowed to use the constructor and methods (and the class at all), we should use the **public** keyword, which is also an access modifier:

```
class Person {  
    private String name;  
    private int age;  
  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    String getName() {  
        return name;  
    }  
  
    int getAge() {  
        return age;  
    }  
  
    void haveBirthday() {  
        age++;  
    }  
}
```

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void haveBirthday() {  
        age++;  
    }  
}
```

'Final' modifier

In the Person class above, there is **no setter** method for the name field, nor is there any other method which changes a person's name after it's been initialised in the constructor. If this is a deliberate design decision, then again we can ask the compiler to enforce this rule by labelling the field with the final keyword:

```
public class Person {  
    private final String name;  
    private int age;  
  
    // ...  
}
```

The **final keyword** goes before the field declaration, but after the access modifier.

After the **name field** has been assigned in the constructor, if we attempt to write a method which assigns it a new value, then we'll get a compilation error. For example, if we forgot our design decision (or another programmer didn't know about it) and wrote a `setName` method, then the class would fail to compile.

If every field in a class is `final`, then its objects are `immutable`.

Why use 'private' and 'final'?

From one perspective, the only purpose of the `private` and `final` fields is to **stop** doing something — accessing a field or a method from outside the class, or changing its value.

1. It documents the intention behind your code, for the benefit of other programmers reading your code. The `private` and `final` keywords **add more meaning** to your code, making it **easier for a reader** to understand.
2. The `private` and `final` keywords tell the compiler to **give you an error** if you use a field in a way you don't intend to -- getting a compilation error.



Design and implement an OOP solution based on a given specification

- Use the specification of the **User Account** class in the *Tasks to Complete – OOP Modelling* Sheet to implement the Class.
- Use the specification of the **Bank Account** class in the *Tasks to Complete – OOP Modelling* Sheet to implement the Class.

```
public class User {
    private final String username;
    private String password;

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public boolean checkPassword(String password) {
        return this.password.equals(password);
    }

    public void setPassword(String oldPassword, String newPassword) {
        if(this.checkPassword(oldPassword)) {
            this.password = newPassword;
            System.out.println("Password changed successfully.");
        } else {
            System.out.println("Incorrect password; not changed.");
        }
    }
}
```

```
public class BankAccount {
    private final String accountNumber;
    private final User accountHolder;
    private double balance;

    public BankAccount(String accountNumber, User accountHolder) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = 0;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public User getAccountHolder() {
        return accountHolder;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if(amount > 0) {
            balance += amount;
            System.out.println("Deposit completed successfully.");
        } else {
            System.out.println("Invalid amount.");
        }
    }

    public void withdraw(double amount) {
        if(amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawal completed successfully.");
        } else {
            System.out.println("Invalid amount.");
        }
    }
}
```